



ME3297 – Applications of Artificial Intelligence in Supply Chain Management

Major Assignment – Final Report

Section L __ Team __

Flagging Orders at Risk of Late Delivery

DataCo Smart Supply Chain – a decision at order
release

Advisor(s): Ngô Đình Phương

Student(s): Lương Trương Quang Huy 2352385

Lê Tuấn Tài 2312997

Nguyễn Quốc Đăng Khoa 2311625

Nguyễn Lê Chí Kiên 2352643

Phạm Hải Nam 2352783

Lê Thanh Duy 2352178



Contents

1	Executive summary	1
2	Problem and decision	1
2.1	Supply-chain situation	1
2.2	Decision, decision-maker and moment	1
2.3	Target and what is known at the moment of decision	2
2.4	Baselines the models must beat	2
2.5	Cost of each kind of error	2
3	Data	3
3.1	Source and licence	3
3.2	Structure and size	3
3.3	Quality assessment	3
3.4	Is the outcome operational? A realism check	3
4	Data preparation	5
5	Method	6
5.1	From scores to decisions	6
6	Validation design	7
7	Results	7
7.1	Cross-validation on the training period	7
7.2	Held-out test period (2017-07 to 2018-01)	9
7.3	Is there any signal beyond shipping mode?	9
8	Interpretation	9
9	Recommendation	11
9.1	Action 1: fix the promises (largest effect)	11
9.2	Action 2: flag orders at release with the lookup table	11
10	Limitations	13
11	Contribution table	13
A	Data dictionary	15
B	Model settings and tuning	16
C	Additional results	17
D	Reproducibility and code excerpts	18
E	Declaration of AI assistant use	20

1 Executive summary

Recommendation. DataCo’s late deliveries are mostly a promise-setting problem, not an order-selection problem. We recommend three actions.

1. **Change two promises first.** First Class is promised in 1 day but every First Class order takes exactly 2 days, so all of them are late. Same Day orders placed after 12:00 always arrive the next day. Promising 2 days for First Class, and next-day delivery for afternoon Same Day orders, cuts late orders from **246 to 169 per week** (-31%). The average promise lengthens by only 0.18 days.
2. **At order release, flag orders with a five-row lookup table, not a machine-learning model.** Flag an order when $P(\text{late}) \times \text{order value} > \200 . This flags about 187 of 428 orders per week, catches 129 late ones and saves about **\$2,400 per week**, against \$1,500 for the best fixed rule.
3. **Re-check on real delivery records before deploying.** The delivery timings in this dataset follow deterministic, apparently generated patterns.

Evidence. We trained four baselines and six models (decision tree, random forest, gradient boosting, SVM, MLP and logistic regression) on 49,826 orders from 2015–2017-06, and tested them once on 13,071 later orders. The best models, the decision tree (PR-AUC 0.854 ± 0.005 over 5 rolling-origin folds) and gradient boosting (0.853 ± 0.003), tie with the lookup table (0.855 ± 0.002). Within Second and Standard Class the models rank orders at chance level (ROC-AUC 0.47–0.52), and across 36 cost scenarios the best model and the table stay within \$25 per week of each other. Shipping mode and order hour carry the entire signal.

2 Problem and decision

2.1 Supply-chain situation

DataCo Global sells clothing, sporting goods and electronics online to 164 countries. At checkout the customer chooses a shipping mode, and each mode carries a fixed promise: Same Day (0 days), First Class (1), Second Class (2) or Standard Class (4). In the data, 57% of the orders that were not cancelled arrived late. Each late order costs goodwill credits, repeat contacts and, eventually, customers.

2.2 Decision, decision-maker and moment

- **Decision.** Should this order be treated as high-risk before it leaves the warehouse? If so, it gets priority handling or a mode upgrade, or customer service contacts the buyer in advance with a realistic date.
- **Decision-maker.** The fulfilment planner, with customer service acting on the flagged list.

- **Moment.** Order release: the order has been placed and paid, but not yet dispatched. At that moment the planner knows the order contents, prices and discounts, the customer and destination, the chosen mode and its promise, and the time of day. The planner does *not* know the actual transit time or the final order status.

2.3 Target and what is known at the moment of decision

The target is `Late_delivery_risk` ($1 = \text{late}$). We verified that for every order that was not cancelled it equals `Days for shipping (real) > Days for shipment (scheduled)`. Actual transit takes 0–6 days, and we assume the outcome is recorded one day after delivery, so the label becomes known 1–7 days after the order. Section 4 lists the columns we excluded because they are only known after the outcome.

2.4 Baselines the models must beat

The dataset does not describe DataCo’s current practice, so we built baselines of increasing strength:

1. **Majority class:** treat every order as late (57% of orders are late).
2. **Rule A**, our proxy for current practice: flag every First and Second Class order, because these modes carry the tightest promises.
3. **Rule B:** Rule A, plus Same Day orders placed at or after a cut-off hour learned from the training data.
4. **Lookup table:** the historical late share for each mode, with Same Day split at the cut-off hour. This uses the same information as Rule B but outputs a probability.

A model only adds value if it beats the lookup table.

2.5 Cost of each kind of error

We assume the following costs (Table 2.1). The dataset contains none of them, so Section 9 tests 36 alternative combinations.

Table 2.1: Cost assumptions per order (USD). v is the order value; the median is about \$560 in the training period.

Outcome	Cost	Reasoning
False alarm (flagged, on time)	\$10	priority handling, an upgrade or a call that was not needed
Missed late order	$0.10 v$ ($\approx \$56$)	goodwill credit, repeat contacts, churn risk
Caught late order	$\$10 + 0.05 v$	action cost; half of the late-delivery cost is avoided
Correctly not flagged	0	–

At the median order value, a missed late order costs about five times as much as a false alarm. For cheap orders the ratio falls below one, which drives the policy in Section 9.

3 Data

3.1 Source and licence

We use *DataCo Smart Supply Chain for Big Data Analysis*, Version 5, by Constante, Silva and Pereira^[3] (Route A, approved dataset no. 1). It is published on Mendeley Data under a CC BY 4.0 licence and cited as the publisher requires. We use only the main table, `DataCoSupplyChainDataset.csv`, and its column description file. The clickstream log in the same archive is not used.

3.2 Structure and size

The file holds **180,519 order lines** $\times$ **53 columns**: 65,752 orders (2.7 lines each on average) from 20,652 customers, covering 118 products, placed between 2015-01-01 and 2018-01-31. Every line of an order carries the same label and the same order-level attributes (the pipeline checks this), so we model at the *order* level, where the decision is taken. After removing 7,754 cancelled lines (Section 4), **62,897 orders** remain. Appendix A gives the data dictionary for all 28 features used.

3.3 Quality assessment

- **Missing values** are rare, and only in columns we do not use: **Product Description** is 100% empty, **Order Zipcode** is 86.2% empty, **Customer Lname** is missing in 8 rows and **Customer Zipcode** in 3. No feature we use has a missing value.
- **Outliers** were counted with the $1.5 \times \text{IQR}$ rule. We kept them, because expensive orders and heavy losses are real business events. Flagged values: line **Sales** 0.3%, product price 1.1%, and **Benefit per order** 10.5% (range $-\$4,275$ to $+\$912$).
- **Class balance and concentration.** 57.3% of orders are late, so predicting “late” for every order already scores 57% accuracy. By mode (Figure 3.2): First Class 100%, Second 80%, Same Day 48%, Standard 40%.
- **Drift.** From 2017-Q4, orders shrink from about 3.0 lines to 1.0 line and the active catalogue falls from about 55 to 10 products. The median order value drops from about \$600 to \$85 (2018-Q1). Order counts and the late share stay stable (Figure 3.1).

3.4 Is the outcome operational? A realism check

Because the label is so concentrated, we tested whether transit time depends on anything real. The results (Table 3.1) show it does not. Transit days are a deterministic function of fields

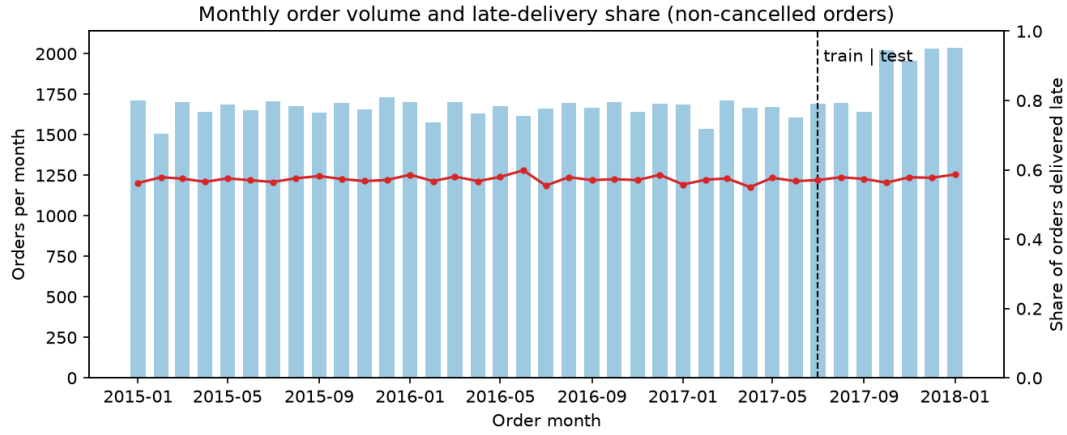
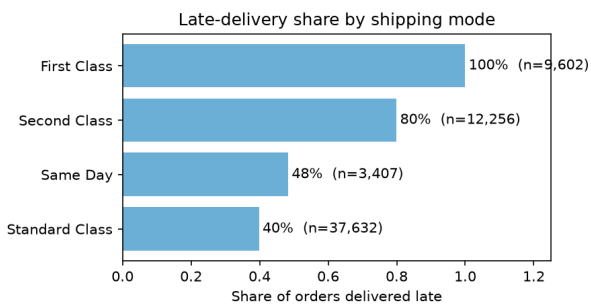
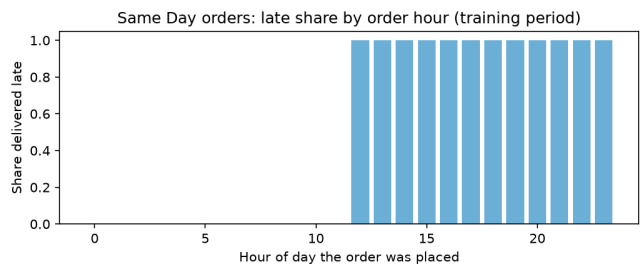


Figure 3.1: Orders per month (bars, left axis) and share delivered late (line, right axis), non-cancelled orders. The dashed line marks the train/test split.



(a) Late share by shipping mode (all periods).



(b) Same Day late share by order hour (training period).

Figure 3.2: The label is concentrated by shipping mode. Within Same Day it depends only on the hour of ordering.

with no operational meaning, and order timestamps are spaced in multiples of 21 minutes. The delivery timings in this dataset were almost certainly generated by an algorithm. The dataset is on the approved list, so we keep it, but we treat this finding as a central limitation (Section 10). It is also the reason we compare every model against the lookup table rather than only against a weak rule.

Table 3.1: Realism checks on the 62,897 orders that were not cancelled.

Check	Share of orders matching
Standard and Second Class: $\text{real days} = (\text{Order Id} - 1) \bmod 5 + 2$	100.0%
First Class: $\text{real days} = 2$	100.0%
Same Day: $\text{late} \iff \text{order placed at or after 12:00}$	100.0%
Gap between consecutive orders is a multiple of 21 minutes	97.5%

4 Data preparation

Table 4.1 lists every preparation decision. All of them are implemented in `src/data_prep.py` and in the scikit-learn pipeline.

Table 4.1: Preparation decisions, their justification, and what each one risks.

Decision	Why, and what it risks
Drop 7,754 cancelled or suspected-fraud lines	These orders are never delivered, so there is no delivery decision for them. We assume the cancellation or fraud check happens before dispatch.
Aggregate lines to orders	The decision is per order. Keeping lines would count multi-line orders several times and let lines of one order fall on both sides of a CV split.
Order-level features	Lines, quantity, value, discount, maximum unit price, number of categories, profit, margin, and the department and category of the highest-value line.
Calendar features	Weekday, hour and month of the order. The raw timestamp and <code>Order Id</code> are <i>not</i> used, because they encode the order sequence, and the sequence determines the outcome here (Table 3.1).
History features	Prior order count and smoothed prior late rate, $(\text{late} + 2.5)/(n + 5)$, per customer, city, and city $\times$ mode. A past order counts only once its outcome was known.
Drop PII and empty columns	Email, password, names, street and zip codes; <code>Product Description</code> , <code>Order Zipcode</code> and <code>Product Image</code> .
Keep outliers	They are genuine prices and losses, and trees are insensitive to them.
Encoding and scaling	One-hot encoding for 6 low-cardinality categoricals (levels with fewer than 20 orders are pooled). Target encoding ^[6] , cross-fitted over 5 folds, for country (164), city (3,585) and category (50). Median imputation. Standard scaling for non-tree models only.

Leakage risks specific to this dataset and how each is avoided.

1. `shipping date` (`DateOrders`) looks like a dispatch date. In fact it equals the order date plus the *real* transit days in 97.4% of rows, so it encodes the outcome. It is excluded, together with `Days for shipping (real)`, `Delivery Status` and `Order Status`.

2. **Order sequence.** Transit days follow `Order Id` modulo 5, so a model given the order ID or the raw timestamp could learn how the data was generated. Neither is a feature, and `run_all.py` stops if one is ever added.
3. **History features.** A naive “past late rate” would include orders that had not yet been delivered. We count a past order only after its order date + real transit days + 1 day.
4. **Preprocessing.** Imputers, scalers and encoders sit inside the `Pipeline` and are refit on the training part of every fold. Target encoding is also cross-fitted within the training data.

5 Method

We compare six models from four families. Three of them (trees, SVM and neural networks) are taught in Weeks 6–13. The four baselines from Section 2 are not counted as model families. All models share the same features and the same preprocessing pipeline, built with `scikit-learn`^[7]. Appendix B gives the full settings.

Table 5.1: Models compared and the reason each was chosen.

Model	Family	Why it is here
Logistic regression	linear	Transparent reference; if it matches the others, the signal is simple.
Decision tree (depth 6)	trees (Wk 6)	Rules a planner can check; depth from a sweep (Appendix B).
Random forest (300 trees)	trees, bagging	Interactions such as city × mode, lower variance ^[2] .
Gradient boosting	trees, boosting	Strongest tabular learner ^[5] ; finds any weak signal.
SVM, RBF kernel	SVM (Wk 8)	Margin-based bias; 600-component Nyström map ^[9] + linear SVM, since exact RBF is $O(n^2)$.
MLP (64–32)	ANN (Wk 7)	Smooth non-linear effects; L2 penalty, early stopping.

5.1 From scores to decisions

A score by itself is not a decision, so every method is turned into a flagging *policy*:

- **Probabilistic methods** (lookup table, logistic regression, trees, MLP) use the expected-value rule^[4]: flag an order when the late-delivery cost it is expected to avoid exceeds the action cost,

$$\hat{p} \cdot \underbrace{0.5}_{\text{avoided}} \cdot \underbrace{0.10 v}_{\text{miss cost}} > \underbrace{\$10}_{\text{action}} \iff \hat{p} \cdot v > \$200. \quad (5.1)$$

No threshold is tuned, but the rule needs calibrated probabilities, so we report the Brier score.

- **SVM** scores are not probabilities. Its threshold is the one that maximises saving on out-of-fold training scores.
- **Rules** flag exactly the orders they select. The majority baseline flags every order.

The cost model then turns each policy into dollars per week (Table 2.1).

6 Validation design

Split. The data is split **by time and never shuffled**. Orders placed before 2017-07-01 form the training set (49,826 orders, 57.2% late). Orders from 2017-07-01 to 2018-01-31 form the test set (13,071 orders, 57.6% late, 30.6 weeks). Nothing chosen during development used the test set. That includes model settings, the SVM threshold, the Same Day cut-off hour and the choice of best model. The test set was scored once, at the end.

Cross-validation. We use rolling-origin validation^[1] with `TimeSeriesSplit(5)` on the training period. Each fold trains on all earlier orders and validates on the next block of about 8,300 orders. Every table reports the mean and standard deviation across the 5 folds. On the test set, 95% intervals come from 200 bootstrap resamples, and all methods share the same resamples.

Metrics and why.

- **PR-AUC (average precision)**, the headline metric: how pure the flagged list is at every alert rate^[8]. scikit-learn scores a block of tied scores at its pooled precision, which penalises few-valued methods (lookup table: 0.805 instead of 0.859 on test), so we break ties at random (mean of 5 draws). Continuous scores are unaffected.
- **ROC-AUC**, also computed within each shipping mode, to isolate any signal beyond what the rules capture. **Recall and precision at a 35% alert rate**, roughly the capacity implied by Rule A.
- **Net saving per order and per week** under Table 2.1, the metric tied to the decision. The **Brier score** checks calibration for the expected-value policy.

Accuracy is reported only alongside these metrics.

7 Results

7.1 Cross-validation on the training period

All six models beat the majority class and both rules; none beats the lookup table. The gaps between the table, the decision tree and gradient boosting (0.001–0.002) are below one fold SD, so they are tied. The decision tree is formally the best model by mean CV PR-AUC and is carried forward. Folds are very stable (Figure C.1 in Appendix C), which is expected when the label depends on only two fields.

Table 7.1: Rolling-origin CV, 5 folds: mean $\pm$ standard deviation across folds. PR-AUC is computed with ties broken at random. Recall and precision are measured at a 35% alert rate. Best value in bold.

Method	PR-AUC	ROC-AUC	Recall@35%	Prec.@35%	Brier
Majority class	0.573 ± 0.003	0.500 ± 0.000	0.348 ± 0.007	0.569 ± 0.010	0.245
Rule A: First/Second	0.790 ± 0.004	0.724 ± 0.003	0.540 ± 0.002	0.884 ± 0.005	–
Rule B: + Same Day cut-off	0.806 ± 0.003	0.745 ± 0.002	0.546 ± 0.003	0.894 ± 0.004	–
Lookup table	0.855 ± 0.002	0.774 ± 0.002	0.550 ± 0.002	0.901 ± 0.004	0.175
Logistic regression	0.837 ± 0.002	0.743 ± 0.003	0.542 ± 0.001	0.886 ± 0.004	0.190
Decision tree	0.854 ± 0.005	0.770 ± 0.006	0.550 ± 0.002	0.901 ± 0.003	0.180
Random forest	0.850 ± 0.003	0.769 ± 0.004	0.549 ± 0.003	0.899 ± 0.002	0.179
Gradient boosting	0.853 ± 0.003	0.771 ± 0.004	0.551 ± 0.003	0.901 ± 0.002	0.179
SVM (RBF, Nyström)	0.840 ± 0.005	0.753 ± 0.010	0.546 ± 0.003	0.893 ± 0.005	–
MLP	0.845 ± 0.007	0.761 ± 0.011	0.544 ± 0.006	0.891 ± 0.008	0.186

Table 7.2: Test-set results, with 95% bootstrap intervals (200 resamples) in brackets. Policies are defined in Section 5.1. Saving is measured against taking no action, in USD per order.

Method	PR-AUC	ROC-AUC	Alert rate	Recall / Prec.	Saving/order
Majority (flag all)	0.577 [0.567, 0.585]	0.500	100%	1.00 / 0.58	2.31 [1.94, 2.57]
Rule A	0.794 [0.784, 0.802]	0.724	35%	0.54 / 0.89	3.23 [3.01, 3.41]
Rule B	0.813 [0.803, 0.821]	0.750	38%	0.59 / 0.90	3.51 [3.27, 3.69]
Lookup table	0.859 [0.853, 0.864]	0.777	44%	0.53 / 0.69	5.61 [5.30, 5.86]
Logistic regression	0.845 [0.838, 0.852]	0.757	44%	0.51 / 0.67	5.48 [5.18, 5.73]
Decision tree	0.861 [0.855, 0.867]	0.780	44%	0.53 / 0.69	5.60 [5.29, 5.84]
Random forest	0.858 [0.851, 0.864]	0.782	45%	0.53 / 0.68	5.58 [5.28, 5.84]
Gradient boosting	0.860 [0.854, 0.867]	0.781	44%	0.53 / 0.69	5.62 [5.32, 5.87]
SVM (RBF, Nyström)	0.850 [0.842, 0.856]	0.770	99%	1.00 / 0.58	2.39 [2.04, 2.65]
MLP	0.858 [0.851, 0.864]	0.778	42%	0.51 / 0.69	5.56 [5.25, 5.81]

7.2 Held-out test period (2017-07 to 2018-01)

The test results repeat the CV ranking, with overlapping intervals among the lookup table and the tree models. The expected-value policies save about \$5.6 per order, against \$3.5 for the best rule. They flag *fewer* late orders than the rules (recall 0.53 against 0.59) but choose them by value: a cheap First Class order is not worth a \$10 action. The SVM threshold chosen on training folds flags almost everything, and so it gives up most of the saving. This is a known weakness of models without calibrated probabilities. Figure C.2 in Appendix C shows the precision–recall curves.

7.3 Is there any signal beyond shipping mode?

Table 7.3 answers the key question. Inside Second Class and Standard Class, which together hold 79% of test orders, every model ranks orders at chance level. The only within-mode signal is Same Day, and the models find it only because they rediscover the noon cut-off. First Class has no variation to explain.

Table 7.3: Test ROC-AUC computed within each shipping mode (0.5 = chance). First Class is always late, so its AUC is undefined.

Mode	Orders	Late	LogReg	Tree	Gr. boost.	MLP
Same Day	731	54%	0.903	1.000	1.000	0.999
Second Class	2,586	81%	0.499	0.478	0.471	0.501
Standard Class	7,780	39%	0.519	0.511	0.513	0.509
First Class	1,974	100%	–	–	–	–

Drift check. When the test period is split at 2017-10-01 (Table C.2), PR-AUC holds for every method, but the saving per order falls from \$9.8 to \$3.0 as the median order value falls from \$610 to \$253. The ranking is stable; the dollar value is not.

8 Interpretation

What drives the model. We measured permutation importance on the test set as the drop in PR-AUC when one feature is shuffled (Figure 8.1). For the decision tree, three features carry almost everything: shipping mode (0.164), scheduled days (0.087, which is the same information as mode) and order hour (0.053). Every other feature scores below 0.001. For gradient boosting the picture is the same: mode 0.283, hour 0.035, and nothing else above 0.002. Destination, customer history, order value, discount and product category do not matter.

Tree structure. The depth-3 tree in Figure 8.2 reads as a policy manual. The first split separates Standard Class (40% late) from the rest. On the left, the tree isolates Same Day

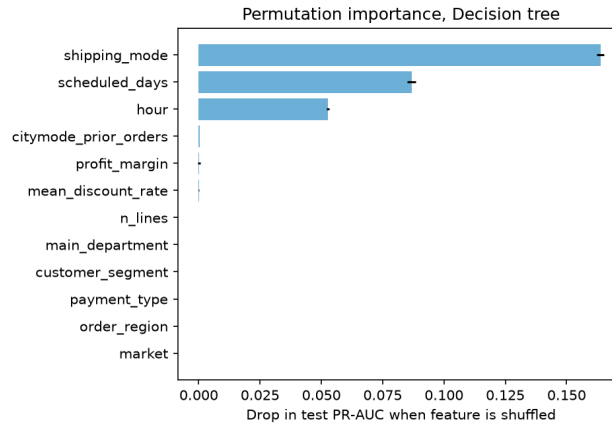


Figure 8.1: Permutation importance for the decision tree: mean drop in test PR-AUC over 5 shuffles, with error bars showing one standard deviation.

($\text{scheduled_days} \leq 0.5$) and splits it at $\text{hour} \leq 11.5$ into leaves that are 0% and 100% late. First Class becomes a 100%-late leaf, and Second Class a leaf that is 80% late. On the right, the splits on target-encoded city and city $\times$ mode history move the late share only between 39% and 43%. That is noise, and these leaves disappear from the importance ranking.

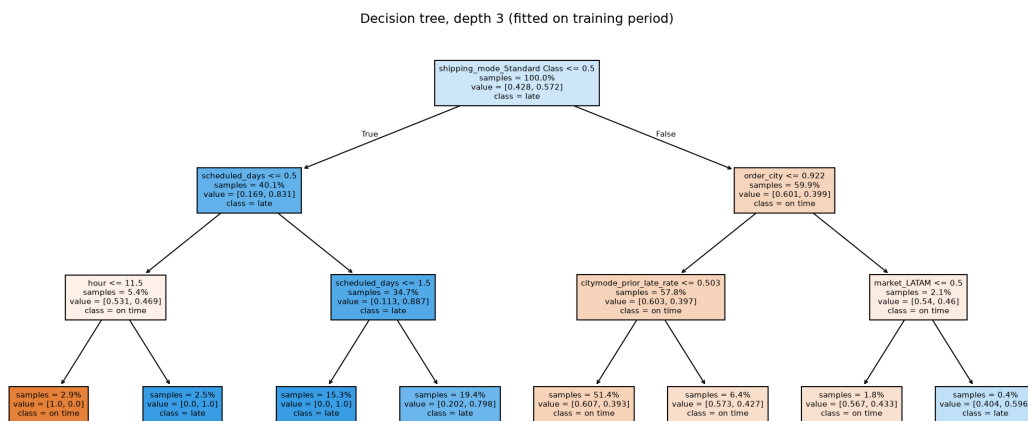


Figure 8.2: Decision tree limited to depth 3, fitted on the training period. **value** gives the shares [on time, late]; **samples** is the share of training orders reaching each node.

Does this make supply-chain sense? Mostly, yes, and it points at the promise rather than the order.

- **First Class** is promised 1 day and always takes 2: a service-level design error, not a risk to predict.
- **Second Class and Standard** both take 2–6 days uniformly, mean 4.0 (Table C.1). Second Class is *not faster*, only promised as faster, and its buyers are late 80% of the time.
- **Same Day** behaves like a warehouse with a 12:00 cut-off, which real fulfilment centres use; the fix is to show the cut-off at checkout.

- The **absence** of destination, customer or product effects is not plausible for real logistics. It is consistent with the realism checks (Section 3.4), so we read it as a property of the data.

The partial-dependence plots in Appendix C confirm this. The average predicted late probability jumps by about 6 points at 12:00. For order value, discount and city $\times$ mode history it moves by at most 2 points, and a feature whose permutation importance is essentially zero cannot turn that movement into better rankings.

9 Recommendation

9.1 Action 1: fix the promises (largest effect)

The cheapest intervention is to change the promise. Table 9.1 applies cumulative changes to the test-period orders, using the noon cut-off learned on the training period.

Table 9.1: Promise scenarios on the test period (428 orders per week). Late cost is $0.10 \times$ order value per late order.

Scenario	Late share	Late/wk	Mean promise	Late cost/wk
Current (0/1/2/4 days)	57.6%	246	2.93 d	\$10,520
A: Same Day after 12:00 promised next day	54.5%	233	2.96 d	\$10,020
B: A + First Class promised 2 days	39.4%	169	3.11 d	\$7,240
C: B + Second Class promised 4 days	31.5%	135	3.50 d	\$5,770
D: C + Second/Standard promised 6 days	0.0%	0	5.09 d	\$0

Recommended: scenario B, now. First Class already takes exactly 2 days, so the new promise costs no speed, and a checkout cut-off for Same Day is standard practice. B removes 77 late orders and about \$3,300 of late cost per week. Scenario C is a commercial decision: Second Class is no faster than Standard, so DataCo should fix it or stop selling it as faster. We do not recommend D, whose lost conversions the data cannot measure.

9.2 Action 2: flag orders at release with the lookup table

Some late orders remain after the promise change, so the planner still needs a daily list. Figure 9.1 plots weekly net saving against the share of orders flagged, with orders ranked by $\hat{p} \times v$. The saving peaks at about a 45% alert rate for both the lookup table and the decision tree (\$2,400 per week). Beyond that point, each extra check costs more than the late delivery it prevents. Rule B peaks earlier, at 25% (\$1,810 per week), because it cannot tell a Second Class order (80% late) from a First Class one (100% late).

Recommended policy: each morning, flag the orders for which $P(\text{late}) \times \text{order value} > \200 , where $P(\text{late})$ comes from the five-row lookup table below. The expected workload is about **190 orders per week**, roughly 27 per day. That is about 16 hours of customer-service

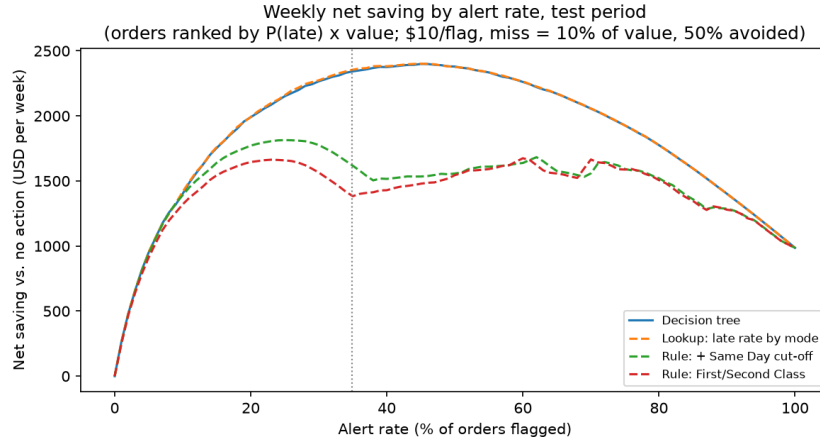


Figure 9.1: Weekly net saving compared with taking no action, by alert rate, on the test period. The dotted line marks Rule A’s alert rate.

Table 9.2: Weekly operational effect of each policy on the test period (428 orders per week).

Policy	Flagged	Late caught	Late missed	False alarms	Net saving
Lookup table, EV rule	187	129	117	57	\$2,399
Decision tree, EV rule	188	129	117	59	\$2,393
Rule B (+ Same Day cut-off)	162	146	101	16	\$1,503
Rule A (First/Second)	149	133	113	16	\$1,381
Flag every order	428	246	0	181	\$986

time per week, assuming 5 minutes per order. It catches 129 late orders per week and saves about \$900 per week more than Rule B.

Cell (from training data)	$P(\text{late})$	Flag if order value exceeds
First Class; Same Day after 12:00	1.00	\$200
Second Class	0.80	\$250
Standard Class	0.40	\$500
Same Day before 12:00	0.00	never

Why the lookup table and not the model. The tree and gradient boosting give the same answer at extra cost: retraining, monitoring, and logic planners cannot check by eye. Across all 36 cost settings (action \$2–\$20, miss 5–20% of value, 30–70% avoided), the best model and the table differ by −\$24 to +\$5 per week, while the table beats Rule B by \$300–\$5,500 per week. At a \$2 action cost the optimal alert rate rises to 60–90%.

Size of the effect. Under the stated assumptions, scenario B avoids about **\$3,300 per week** of late-delivery cost, and the flagging policy saves a further **\$900 per week** over the current-practice proxy: roughly \$200k per year at 2017 volumes. Because the saving scales with order value, which fell by two-thirds in late 2017, we would quote it as “tens to low hundreds of thousands of dollars per year”.

10 Limitations

What the model cannot do. It cannot say *which* Standard or Second Class orders (79% of volume) will be late; within those modes it is no better than a coin. It does not predict the *size* of a delay.

Where the data is unrepresentative.

- **Synthetic outcome.** Transit days follow the order ID modulo 5 (Table 3.1). “Only mode and hour matter” may describe the data generator, not DataCo’s network; on real data, destination, carrier and warehouse load would very likely carry signal. The pipeline is ready for such data, but the “use the lookup table” conclusion may not transfer.
- **Drift after 2017-Q4.** Ranking quality held, but the saving per order fell from \$9.8 to \$3.0.
- **Costs are assumed.** The *choice* of policy is robust across the sensitivity grid; the size of the saving is not.
- **Promise changes affect demand.** Longer promises may reduce conversion, which the data cannot measure. Scenario B only aligns promises with what already happens; C and D need a controlled test.

What must be true for the recommendation to hold. Real transit times depend mainly on mode and cut-off; customers accept a 2-day First Class promise; a flagged order avoids about half of the late-delivery cost.

With more time or data. We would refit the pipeline on carrier and warehouse records (dispatch times, carrier, distance), A/B-test the promise change in one market, estimate the effect of an action from past interventions instead of assuming 50%, and add a regression model for the size of the delay.

11 Contribution table

Table 11.1: Lead responsibilities.

Member	ID	Led
Lê Thanh Duy	2352178	Problem framing, cost model, recommendation (Sections 2 and 9)
Lương Trương Quang Huy	2352385	Data audit, quality and realism checks (Section 3)
Lê Tuấn Tài	2312997	Order-level features, leakage control, history features (Section 4)
Nguyễn Quốc Đăng Khoa	2311625	Validation design, time split, bootstrap intervals (Section 6)
Nguyễn Lê Chí Kiên	2352643	Tree models and interpretation (Sections 5 and 8)
Phạm Hải Nam	2352783	SVM and MLP models, results tables, presentation (Section 7)

References

- [1] Christoph Bergmeir and José M. Benítez. On the use of cross-validation for time series predictor evaluation. *Information Sciences*, 191:192–213, 2012.
- [2] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- [3] Fabian Constante, Fernando Silva, and António Pereira. DataCo SMART SUPPLY CHAIN FOR BIG DATA ANALYSIS. Mendeley Data, Version 5. Licence CC BY 4.0, 2019. URL <https://data.mendeley.com/datasets/8gx2fvg2k6/5>.
- [4] Charles Elkan. The foundations of cost-sensitive learning. In *Proceedings of the 17th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 973–978, 2001.
- [5] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001.
- [6] Daniele Micci-Barreca. A preprocessing scheme for high-cardinality categorical attributes in classification and prediction problems. *ACM SIGKDD Explorations Newsletter*, 3(1):27–32, 2001.
- [7] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [8] Takaya Saito and Marc Rehmsmeier. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLoS ONE*, 10(3):e0118432, 2015.
- [9] Christopher K. I. Williams and Matthias Seeger. Using the Nyström method to speed up kernel machines. In *Advances in Neural Information Processing Systems 13*, pages 682–688, 2001.

A Data dictionary

All 28 features used, plus the target, at order level (62,897 orders). No feature has missing values after preparation. The raw source columns are described in the column description file shipped with the dataset.

Table A.1: Data dictionary of the order-level table.

Feature	Type	Description	Unique
shipping_mode	categorical	Shipping mode chosen at order (Standard/Second/First/Same Day)	4
market	categorical	Destination market (5 values)	5
order_region	categorical	Destination region (23 values)	23
customer_segment	categorical	Consumer / Corporate / Home Office	3
payment_type	categorical	Transaction type: DEBIT, TRANSFER, PAYMENT, CASH	4
main_department	categorical	Department of the order's highest-value line	11
order_country	categorical, target-encoded	Destination country	164
order_city	categorical, target-encoded	Destination city	3,585
main_category	categorical, target-encoded	Product category of the highest-value line	50
scheduled_days	numeric	Promised shipping days (fixed by mode: 0/1/2/4)	4
n_lines	numeric	Number of order lines	5
total_qty	numeric	Total units ordered	24
order_value	numeric, USD	Sum of line Sales (before discount)	18,647
total_discount	numeric, USD	Sum of line discounts	20,182
mean_discount_rate	numeric	Mean line discount rate	4,513
max_unit_price	numeric, USD	Highest unit price in the order	75
n_categories	numeric	Distinct product categories in the order	5
order_profit	numeric, USD	Sum of 'Benefit per order' over lines	52,398
profit_margin	numeric	order_profit / order_value	58,088
weekday	numeric	Day of week of order (0 = Monday)	7
hour	numeric	Hour of day of order	24

Feature	Type	Description	Unique
month	numeric	Month of order	12
cust_prior_orders	numeric	Customer's earlier orders whose outcome was already known	15
cust_prior_late_rate	numeric	Smoothed late share of those orders	72
city_prior_orders	numeric	Earlier orders to the same city with known outcome	633
city_prior_late_rate	numeric	Smoothed late share of those orders	3,653
citymode_prior_orders	numeric	Same, for city x shipping mode	399
citymode_prior_late_rate	numeric	Smoothed late share of those orders	2,485
late	binary target	Late_delivery_risk: 1 if real shipping days > scheduled days	2

B Model settings and tuning

Table B.1: Full parameter settings (scikit-learn 1.9.1, `random_state=42`). Unlisted parameters are library defaults.

Model	Settings
Majority class	<code>DummyClassifier(strategy="prior")</code>
Rule A / Rule B	Custom estimator. Rule B learns its cut-off in <code>fit()</code> as the first hour whose Same Day late share exceeds 50% (12:00 on the training set).
Lookup table	Custom estimator: mean late share per cell (4 modes, with Same Day split at the learned cut-off).
Logistic regression	<code>C=1.0, max_iter=2000</code> ; scaled inputs
Decision tree	<code>max_depth=6, min_samples_leaf=50</code> ; unscaled inputs
Random forest	<code>n_estimators=300, min_samples_leaf=20, max_features="sqrt"</code>
Gradient boosting	<code>HistGradientBoostingClassifier(learning_rate=0.05, max_iter=300, max_leaf_nodes=31, early_stopping=True)</code>
SVM	<code>Nystroem(kernel="rbf", gamma=0.02, n_components=600) → LinearSVC(C=0.1)</code> ; scaled inputs
MLP	<code>hidden_layer_sizes=(64, 32), alpha=1e-3, learning_rate_init=1e-3, max_iter=200, early_stopping=True</code> ; scaled inputs
Preprocessing	<code>OneHotEncoder(min_frequency=20, handle_unknown="infrequent_if_exist"); TargetEncoder(target_type="binary", cv=KFold(5, shuffle=True)); SimpleImputer(median); StandardScaler (non-tree models only)</code>

Table B.2: Decision-tree depth sweep (rolling-origin CV, 5 folds, standard average precision).
Most of the gain with depth comes from deeper trees splitting tied leaves into many small ones, which standard AP rewards (Section 6); with ties broken, depth 6 already scores 0.854.
We kept depth 6 for readability.

max_depth	2	3	4	6	8	12	none
PR-AUC mean	0.760	0.808	0.818	0.832	0.839	0.846	0.847
PR-AUC SD	0.008	0.005	0.010	0.009	0.006	0.003	0.002

C Additional results

Table C.1: Actual transit days by mode in the training period, and the late share that would result if p days were promised. Current promises are shown in bold.

Mode	Orders	Real days		Late share if promise = p days					
		mean	range	$p = 0$	1	2	3	4	6
Same Day	2,676	0.47	0–1	0.47	0.00	0	0	0	0
First Class	7,628	2.00	2–2	1.00	1.00	0.00	0	0	0
Second Class	9,670	4.00	2–6	1.00	1.00	0.80	0.60	0.40	0.00
Standard Class	29,852	4.00	2–6	1.00	1.00	0.80	0.60	0.40	0.00

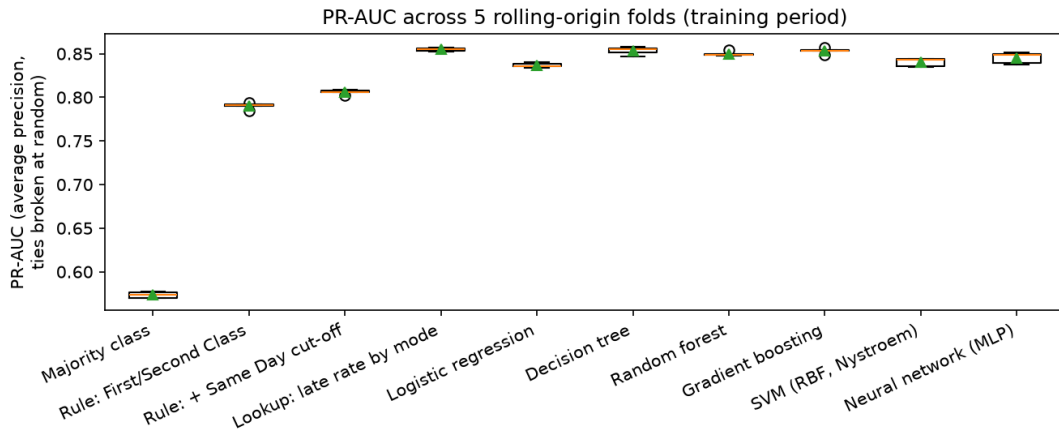


Figure C.1: PR-AUC (ties broken at random) across the 5 rolling-origin folds; triangles mark the mean.

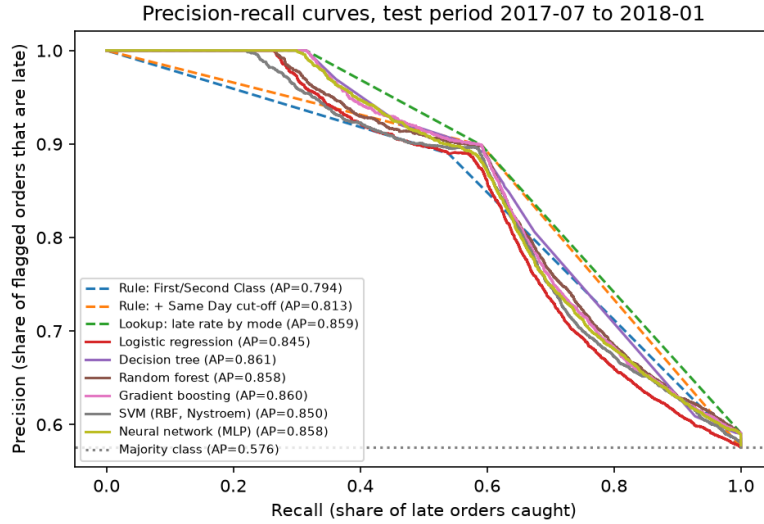


Figure C.2: Precision–recall curves on the test period. Dashed lines are baselines; the AP values in the legend are computed with ties broken at random.

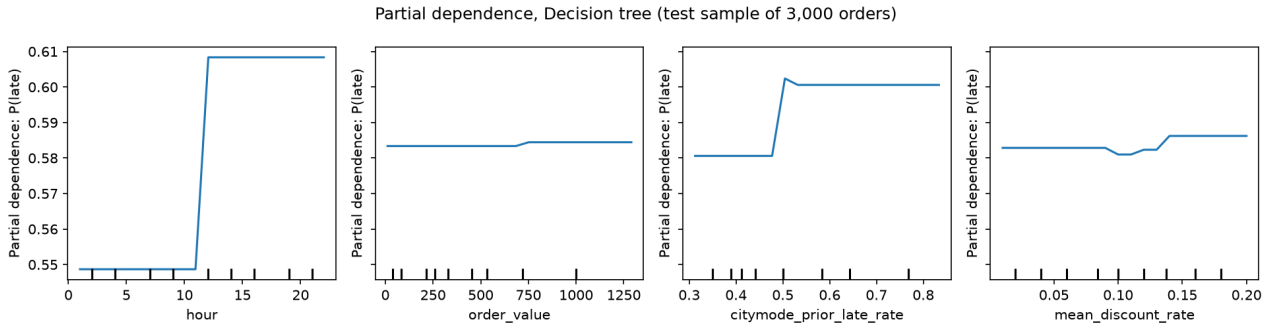


Figure C.3: Partial dependence of the predicted late probability, decision tree, on a test sample of 3,000 orders.

Table C.2: Drift check: the test period split at 2017-10-01. Saving is in USD per order under the expected-value policy.

Period	Method	Orders	Median	PR-AUC	ROC-AUC	Saving/order
2017-07..09	Decision tree	5,028	\$610	0.860	0.778	9.77
	Lookup table			0.859	0.779	9.78
	Rule B			0.814	0.753	7.28
2017-10..2018-01	Decision tree	8,043	\$253	0.862	0.782	2.99
	Lookup table			0.859	0.776	3.00
	Rule B			0.813	0.749	1.16

D Reproducibility and code excerpts

The code runs from a clean start and reproduces every number in this report. We checked this by rebuilding it in a fresh virtual environment and comparing all 24 output tables. Reposi-

tory: https://github.com/KenKout/ME3297_Project. The raw dataset is not included in the submission; `get_data.py` extracts it from the Mendeley archive.

```
1 python -m venv .venv && source .venv/bin/activate
2 pip install -r requirements.txt          # numpy 2.5.3, pandas 3.0.6,
   scikit-learn 1.9.1, matplotlib 3.11.2
3 python get_data.py path/to/8gx2fvg2k6-5.zip
4 python run_all.py                      # ~4 min; writes
   outputs/tables/*.csv and outputs/figures/*.png
```

Leakage-safe history features (`_prior_history` in `src/data_prep.py`, body only; comments in the code are in Vietnamese). For each order, it counts the earlier orders of the same group whose outcome was already available (`avail` = order date + real transit days + 1 day) at the current order time, and how many of those were late.

```
199 n_prior = np.zeros(len(times))
200 n_late = np.zeros(len(times))
201 order = np.argsort(keys, kind="stable")      # chỉ số sau khi sắp xếp theo
   nhóm
202 k_sorted = keys[order]
203 # bounds: các vị trí mà giá trị nhóm thay đổi -> ranh giới giữa các nhóm.
204 bounds = np.flatnonzero(np.r_[True, k_sorted[1:] != k_sorted[:-1], True])
205 for s, e in zip(bounds[:-1], bounds[1:]):
206     idx = order[s:e]                        # chỉ số (trong bảng gốc)
   của các đơn thuộc nhóm này
207     a_order = np.argsort(avail[idx])         # sắp xếp các đơn trong nhóm
   theo thời điểm biết kết quả
208     a_sorted = avail[idx][a_order]
209     # cum_late[j] = số đơn trễ trong j đơn đầu tiên (theo thứ tự biết kết
   quả).
210     cum_late = np.r_[0, np.cumsum(y[idx][a_order])]
211     # pos[i] = số đơn trong nhóm có avail <= times[i] (side="right" ->
   tính cả dấu bằng).
212     pos = np.searchsorted(a_sorted, times[idx], side="right")
213     n_prior[idx] = pos
214     n_late[idx] = cum_late[pos]
215 return n_prior, n_late
```

The expected-value flagging rule (Equation (5.1)) is a single line in `ev_flag` in `src/modeling.py`:

```
1 return np.asarray(p) * cost.effectiveness * cost.miss_fraction *  
   np.asarray(value) > cost.action_cost
```

E Declaration of AI assistant use

As §9 of the project brief requires, we declare that an AI coding assistant (Kiro) was used substantially in this project. It was used to:

- draft the project proposal from the brief;
- write the analysis code (`get_data.py`, `src/`, `run_all.py`), including the data audit that uncovered the deterministic delivery patterns, and add explanatory comments in Vietnamese;
- run the pipeline, check that it reproduces from a clean environment, and draft this report in L^AT_EX from the generated tables and figures.

[Team to complete before submission: state what the team itself did. For example, which analysis decisions you reviewed or changed, which numbers you re-checked against outputs/tables/, and which sections you rewrote in your own words.]

Every member has read the code and the report and is prepared to explain any line or sentence of it during the presentations.